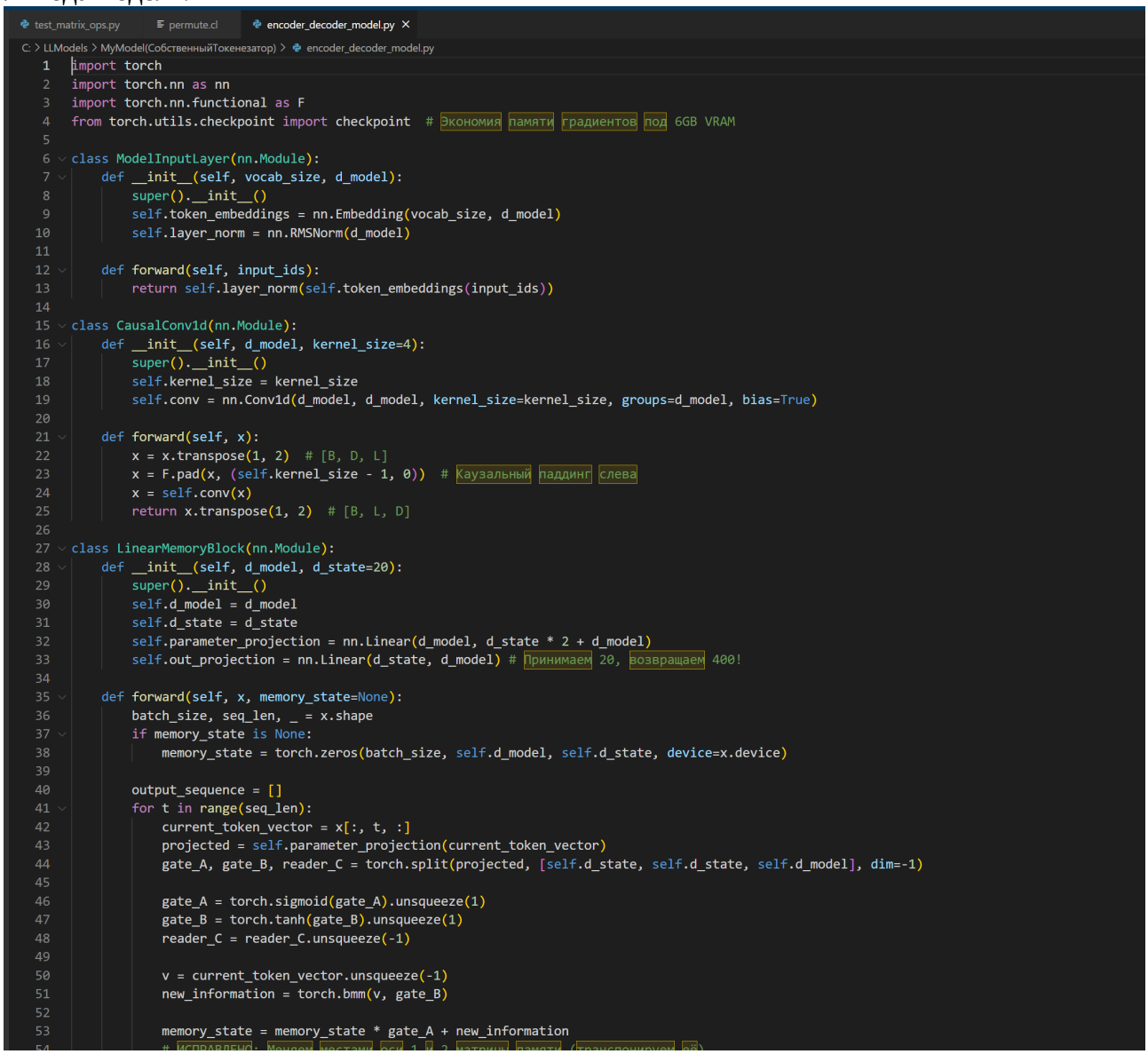
У меня есть идея как организолвать архитектуры лямбда программы (псведокода). Для начала рассмотрим пример python pytorch архитектуры, который потом будем переводить в лямбда модель.



```
            v = current_token_vector.unsqueeze(-1)
            new_information = torch.bmm(v, gate_B)

            memory_state = memory_state * gate_A + new_information
            # ИСПРАВЛЕНО: Меняем местами оси 1 и 2 матрицы памяти (транспонируем её),
            # чтобы размерность 400 совпала с размерностью 400 вектора reader_C
            current_output = torch.bmm(memory_state.transpose(1, 2), reader_C).squeeze(-1)
            output_sequence.append(current_output.unsqueeze(1))

        output_sequence = torch.cat(output_sequence, dim=1)
        return self.out_projection(output_sequence), memory_state

class SwiGLUFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff, bias=False)
        self.w3 = nn.Linear(d_model, d_ff, bias=False)
        self.w2 = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        return self.w2(F.silu(self.w1(x)) * self.w3(x))

class EncoderBlock(nn.Module):
    def __init__(self, d_model=400, d_state=20, dropout=0.1):
        super().__init__()
        self.norm1 = nn.RMSNorm(d_model)
        self.local_conv = CausalConv1d(d_model)
        self.memory = LinearMemoryBlock(d_model, d_state)
        self.dropout1 = nn.Dropout(dropout)
        self.norm2 = nn.RMSNorm(d_model)
        self.ffn = SwiGLUFFN(d_model, d_ff=int(2 * d_model * 4 / 3))
        self.dropout2 = nn.Dropout(dropout)

    def forward(self, x, hidden_state=None):
        residual = x
        normed_x = self.norm1(x)
        conv_x = self.local_conv(normed_x)
        memory_x, new_hidden_state = self.memory(conv_x, hidden_state)
        x = residual + self.dropout1(memory_x)

        residual = x
        ffn_x = self.ffn(self.norm2(x))
        x = residual + self.dropout2(ffn_x)
        return x, new_hidden_state

class DecoderBlock(nn.Module):
    def __init__(self, d_model=400, d_state=20, dropout=0.1):
        super().__init__()
        self.norm1 = nn.RMSNorm(d_model)
        self.memory = LinearMemoryBlock(d_model, d_state)

        self.norm_cross = nn.RMSNorm(d_model)
        self.cross_attention = nn.MultiheadAttention(d_model, num_heads=4, batch_first=True)

        self.norm2 = nn.RMSNorm(d_model)
```

```
        self.memory = LinearMemoryBlock(d_model, d_state)

        self.norm_cross = nn.RMSNorm(d_model)
        self.cross_attention = nn.MultiheadAttention(d_model, num_heads=4, batch_first=True)

        self.norm2 = nn.RMSNorm(d_model)
        self.ffn = SwiGLUFFN(d_model, d_ff=int(2 * d_model * 4 / 3))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, encoder_hidden_states, memory_state=None):
        residual = x
        x_mem, new_memory_state = self.memory(self.norm1(x), memory_state)
        x = residual + self.dropout(x_mem)

        residual = x
        normed_x = self.norm_cross(x)
        attn_out, _ = self.cross_attention(query=normed_x, key=encoder_hidden_states, value=encoder_hidden_states)
        x = residual + self.dropout(attn_out)

        residual = x
        x = residual + self.dropout(self.ffn(self.norm2(x)))
        return x, new_memory_state

class LinearSeq2SeqLLM(nn.Module):
    def __init__(self, vocab_size, d_model=400, d_state=20, num_encoder_layers=10, num_decoder_layers=4, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.input_layer = ModelInputLayer(vocab_size=vocab_size, d_model=d_model)

        # СТРОГАЯ ИМЕНОВАННАЯ ИНИЦИАЛИЗАЦИЯ — защищает пространство имен от перегрузки
        self.encoder_layers = nn.ModuleList([
            EncoderBlock(d_model=d_model, d_state=d_state, dropout=dropout)
            for _ in range(num_encoder_layers)
        ])

        self.decoder_layers = nn.ModuleList([
            DecoderBlock(d_model=d_model, d_state=d_state, dropout=dropout)
            for _ in range(num_decoder_layers)
        ])

        self.final_norm = nn.RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        self.lm_head.weight = self.input_layer.token_embeddings.weight  # Tied-веса для VRAM

    def forward(self, encoder_input_ids, decoder_input_ids, enc_states=None, dec_states=None):
        enc_x = self.input_layer(encoder_input_ids)
        new_enc_states = []

        if enc_states is None:
            enc_states = [None] * len(self.encoder_layers)

        for i, layer in enumerate(self.encoder_layers):
            if self.training:
                enc_x, h_next = checkpoint(layer, enc_x, enc_states[i], use_reentrant=False)
            else:
```

```
            enc_states = [None] * len(self.encoder_layers)

        for i, layer in enumerate(self.encoder_layers):
            if self.training:
                enc_x, h_next = checkpoint(layer, enc_x, enc_states[i], use_reentrant=False)
            else:
                enc_x, h_next = layer(enc_x, enc_states[i])
            new_enc_states.append(h_next)

        dec_x = self.input_layer(decoder_input_ids)
        new_dec_states = []

        if dec_states is None:
            dec_states = [None] * len(self.decoder_layers)

        for i, layer in enumerate(self.decoder_layers):
            if self.training:
                dec_x, h_next = checkpoint(layer, dec_x, enc_x, dec_states[i], use_reentrant=False)
            else:
                dec_x, h_next = layer(dec_x, enc_x, dec_states[i])
            new_dec_states.append(h_next)

        dec_x = self.final_norm(dec_x)
        logits = self.lm_head(dec_x)

        return logits, new_enc_states, new_dec_states
```

Нам будт нужны 2 уровня абстракции:
1- nnclglob будет содержать самостятельные операции (такие как перемножение матриц ,softmax , и другие ияжелые функции которые требуют синхронизации либо расчета данных на строне CPU (к примеру персчитать размер новой матрицы приумножение матриц Y=X*D,D=Y*z1,X=D*W меняя местами указатели на буферы памяти и только перезаписывать буфер размерностей осей (тогда понадобиться всего 3 буфера для всех операций вместо 9 буферов размерностей для выполнения монолитным куском) + внутри глобальных абстракций можно на встроенном уровне добавлять в конец синхронизацию OpenCL чтобы данные были актульны.
Также нам будут нужны два спецефических глобальных объекта:
1. LocalRunner — запускает цепочку лоальных вызовов , преобразуя код в 1 opencl ядро.
2.HostReturner — переводит запрашиваемые данные с gpu на cpu (полезно для отладки).
На уровне LocalRunner мы будем использовать другой псевдокод содержащий nncllоc (сюда входят функции из файла „base_cl_functions“ (синус, косиснус и т. д.). LocalRunner также будет иметь встроенную синхронизацию по завершению выполнения (чтобы гарантировать актуальность данных).
Таким образоми, псевдокод будет выглядеть притмерног так:

```
gate_A, gate_B, reader_C = nnclglob.split(projected, [self.d_state, self.d_state, self.d_model], dim=-1)

nnclglob. LocalRunner(()=>
        get_A=nncllоc.sigmoid(get_A)
        get_B=nncllоc.tanh(get_B)

)
new_information = nnclglob.bmm(v, gate_B)
nnclglob. LocalRunner(()=>
        memory_state=memory_state*get_A
        memory_state=memory_state+new_information
)
tmp=nnclglob.transpose( memory_state,1, 2)
current_output=nnclglob.bmm(tmp,reader_C)
```

Решение проблемы обратного проохода

Чтобы решить проблему обратиного прохода, можно параллельно с основной цепочкой (при включенном реждиме обучения) строить 2 цепочку обратного прохода , заменяя функции их производными.